

Drupal Module Development

Core Concepts

Course Overview

A concise, fast-paced guide to modern Drupal 8/9/10 module development, focusing exclusively on essential concepts and code.

Recommendation: It is highly recommended to read and follow along with **Unit 8: Building a Mini Module**, as it serves as a live, comprehensive example integrating many of the concepts discussed throughout this course.

March 2, 2026

yatlas

Contents

1	Unit 1: The Page Call Process	3
1.1	The Request-Response Lifecycle	3
1.2	The Front Controller: <code>index.php</code>	3
1.3	The Drupal Kernel	3
1.4	Routing and Controllers	3
2	Unit 2: Services and Dependency Injection	4
2.1	What are Services?	4
2.2	The Service Container	4
2.3	Accessing Services	4
2.4	Dependency Injection	4
3	Unit 3: Plugins and the Plugin API	5
3.1	What are Plugins?	5
3.2	Three Essential Concepts	5
3.3	Discovering Plugins via Annotations (PSR-4)	6
4	Unit 4: Creating Your First Module and <code>info.yml</code>	6
4.1	Getting Started	6
4.2	The <code>info.yml</code> File	7
4.3	The <code>.module</code> File	7
5	Unit 5: The Hook System and Implementing Hooks	8
5.1	What are Hooks?	8
5.2	The Three Categories of Hooks	8
5.3	five-step Process for Implementing a Hook	8
5.4	Implementation Example: <code>hook_cron</code>	9
5.5	Implementation Example: <code>hook_form_alter</code>	9
6	Unit 6: Routing and Controllers	10
6.1	The Two-Step Design Approach	10
6.2	Step 1: Building a Controller (PSR-4 Pattern)	10
6.3	Step 2: Wiring it with a Route (<code>routing.yml</code>)	11
6.4	Passing Variables Through Routes	12
7	Unit 7: Menu API and Menu Items	12
7.1	Defining Custom Menu Links	12
7.2	Four Types of Menu Items	13
8	Unit 8: Building a Mini Module (RSVP List)	14
8.1	Sub-unit 1: Setting up the <code>info.yml</code>	14
8.1.1	New <code>info.yml</code> Keys	15
8.1.2	YAML Syntax and Strings	15
8.2	Sub-unit 2: The Form API	16
8.2.1	What is a Drupal Form?	16
8.2.2	Render Arrays	16

8.2.3	Three Common Types of Forms	17
8.2.4	Required Form Methods	18
8.3	Sub-unit 3: Building the Email Submission Form	18
8.3.1	Code Explanation	20
8.4	Sub-unit 4: Building a Route with a Path to a Form	20
8.4.1	Code Explanation	21
8.4.2	Testing the Form	21
8.5	Sub-unit 5: Validating Form Submissions	22
8.5.1	Code Explanation	22
8.6	Sub-unit 6: The Install File and Working with the Database Schema . .	23
8.6.1	The Schema API	23
8.6.2	Creating rsvplist.install	23
8.6.3	Code Explanation	25
8.7	Sub-unit 7: An Introduction to the Database API	25
8.7.1	The Database Connection Object	25
8.7.2	Static Queries vs. Dynamic Queries	26
8.7.3	Handling Results and Errors	27
8.8	Sub-unit 8: Dynamic Insert Queries - Inserting into the Database Pro- grammatically	27
8.8.1	The Three Phases of Form Submission	27
8.8.2	Updating the submitForm Code	28
8.8.3	Validating the Insertion (Database Check)	29
8.9	Sub-unit 9: Defining Custom Permissions - Flexible Access Control . . .	30
8.9.1	Creating permissions.yml	30
8.9.2	Applying the Permission	30
8.10	Sub-unit 10: Block Plugins and Creating the RSVP List Block	31
8.10.1	Creating the Plugin Directory and File	31
8.10.2	Coding the Block Plugin Class	31
8.10.3	Code Explanation	32
8.11	Sub-unit 11: Displaying the RSVPForm in a Block and Applying Access Control	33
8.11.1	Updating the RSVP Block Plugin	33
8.11.2	Code Explanation	34
8.12	Sub-unit 12: Introduction to the Configuration API	35
8.12.1	Configuration vs. State	35
8.12.2	Active Configuration and Storage	35
8.12.3	Configuration Objects	36
8.13	Sub-unit 13: Creating RSVPSettingsForm and Config Settings	37
8.13.1	The ConfigFormBase Class	37
8.13.2	1. Creating the RsvpSettingsForm Classes	37
8.13.3	2. Creating Default Configuration	39
8.13.4	3. Defining the Configuration Schema	39
8.14	Sub-unit 14: Add Routing and Menu Link to RSVPSettings	39
8.14.1	1. Creating the Route	40
8.14.2	2. Creating the Menu Link	40
8.15	Sub-unit 15: Creating an RSVP List Reports Page	42
8.15.1	Creating the ReportController	43
8.15.2	Code Explanation	44

1 Unit 1: The Page Call Process

1.1 The Request-Response Lifecycle

When a server receives a request, Drupal delivers an HTTP response (HTML, JSON, XML, etc.). Drupal is built entirely on the **Symphony framework**, utilizing object-oriented HTTP request and response management.

1.2 The Front Controller: `index.php`

Instead of a separate physical file for every page (legacy approach), Drupal uses `index.php` as a **Front Controller**.

- **Single Entry Point:** All requests (e.g., `/contact`, `/about`) route through `index.php`.
- **Clean URLs:** Web server rewrites hide `index.php` from the final URL path.

1.3 The Drupal Kernel

`index.php` initiates the **Drupal Kernel** (which extends Symfony's HTTP Kernel).

1. A **Request Object** is created representing the client's request.
2. The Request is passed to the Kernel's `handle` method.
3. The Kernel uses the **Routing system** to find the matching module logic.
4. A **Controller** generates and returns a **Response Object**.

1.4 Routing and Controllers

1. Routing (The "Where" - YAML)

The router intercepts the request path and identifies which Controller to execute.

```
my_module.hello_world:
  path: '/helloworld'
  defaults:
    _controller: '\Drupal/my_module/Controller/
      MyHelloWorldController::sayHelloHtml'
```

Listing 1: `my_module.routing.yml`

2. Controllers (The "What" - PHP)

A simple PHP class containing a method that returns a Response.

```
class MyHelloWorldController {
  public function sayHelloHtml() {
    return new Response('<html><body>Hello!</body></html>');
  }
}
```

Listing 2: `MyHelloWorldController.php`

Unit 1 Summary

All requests hit `index.php`, generating a Request object. The Drupal Kernel uses YAML Routes to map the Request to a PHP Controller, which returns the final Response.

2 Unit 2: Services and Dependency Injection

2.1 What are Services?

A **Service** is simply a useful object or class that performs consistent operations for you, such as accessing the database or sending emails. By abstracting these operations into services, Drupal separates business logic from implementation details (e.g., swapping MySQL for SQLite becomes trivial).

2.2 The Service Container

Also known as the **Dependency Injection Container**, this is a single global object (an associative array) that holds, configures, and constructs all available services in Drupal. There are currently over 400 core services (e.g., Database, Logger, Plugin Manager).

2.3 Accessing Services

There are three main methods for accessing services:

1. **Global Static Method (Dedicated):** Used for very common services. E.g., `\Drupal::database()`. Best limited to legacy hook functions.
2. **Generic Service Method:** Used for other services by their machine name. E.g., `\Drupal::service('date.formatter')`.
3. **Dependency Injection (Preferred):** The modern, object-oriented approach where a class receives its required services via its `__construct()` method.

2.4 Dependency Injection

Instead of relying on global state, objects receive the tools they depend on as arguments.

```
class MyServiceClass {
    protected $database;

    // The Container automatically passes the database connection
    service
```

```

    public function __construct(Connection $connection) {
        $this->database = $connection;
    }
}

```

Listing 3: Dependency Injection Example Structure

Unit 2 Summary

Services are helpful objects managed by the Service Container. While you can fetch them globally via `\Drupal::service()`, the preferred approach in modern Drupal is **Dependency Injection**, passing the required services directly into constructors. Custom services are declared in a `services.yml` file.

3 Unit 3: Plugins and the Plugin API

3.1 What are Plugins?

A **Plugin** is a piece of functionality providing a general, reusable solution to a recurring need. By adhering to the **Plugin API** standard, components become swappable and extensible (similar to how any device following the standard can use a USB port).

Examples of common Drupal plugins include:

- **Blocks:** Every custom or core block is a plugin (e.g., `system_powered_by_block`, `system_breadcrumb_block`).
- **Views Displays:** Formats like HTML list, Grid, or Bootstrap tables.
- **Image Actions:** Tools like crop, resize, and rotate (found under the `gd` directory).
- **Migrate Plugins:** Source, Process, and Destination plugins for migrations.

3.2 Three Essential Concepts

1. **Plugin Types:** The base controlling class defining the central purpose of its plugins (e.g., Block, Image Toolkit).
2. **Plugin Discovery:** How Drupal finds available plugins. The four methods are **Static**, **Hook**, **Annotated Class**, and **YAML**. The most common is Annotated Class Discovery.
3. **Plugin Factory:** The design pattern mechanism responsible for actually instantiating the specific plugin object when needed, saving you from writing instantiation boilerplate.

3.3 Discovering Plugins via Annotations (PSR-4)

Because most plugins are discovered via code annotations, Drupal relies heavily on the **PSR-4** autoloading standard.

- **Directory Structure:** Plugins must be placed in exactly: `[module_name]/src/Plugin/[PluginType]`. For example, core block plugins reside in `core/modules/system/src/Plugin/Block`.
- **Annotations:** Placed in the docblock comment directly above the Plugin class. The syntax allows for complex structured data and automatic translation of UI strings.

```
/**
 * Provides a 'Powered by Drupal' block.
 *
 * @Block(
 *   id = "system_powered_by_block",
 *   admin_label = @Translation("Powered by Drupal")
 * )
 */
class SystemPoweredByBlock extends BlockBase { ... }
```

Listing 4: Example: System Powered By Block Annotation

Understanding the Annotation Attributes:

- `@Block` tells Drupal this class is a Block plugin type.
- `id` defines the unique system machine name.
- `admin_label` defines the human-readable text in the UI. By wrapping it in `@Translation()`, this config string is automatically handed off to Drupal's multilingual subsystem.

Unit 3 Summary

Plugins are powerful, swappable components discovered via mechanisms like Annotated Class Discovery. To define a plugin, place your class in the correct PSR-4 `src/Plugin` path, and add the necessary annotation metadata (like `id` and `admin_label`) in the docblock so Drupal's Plugin Factory can find and instantiate it.

4 Unit 4: Creating Your First Module and info.yml

4.1 Getting Started

To create a custom module, you must create a directory for it within your Drupal installation.

- **Location:** Custom modules belong in the `/modules/custom/` directory to separate them from contributed or core modules.
- **Machine Name:** Choose a unique machine name containing only lowercase letters and underscores (e.g., `my_module`). Create a folder with this exact name: `/modules/custom/my_module/`.

4.2 The info.yml File

Every module and theme requires an `info.yml` file to notify Drupal of its existence and provide metadata. Without it, Drupal cannot install the extension. The file must be named `[machine_name].info.yml` (e.g., `mymodule.info.yml`).

Required Keys:

- `name`: The human-readable name displayed on the Extend administration page.
- `type`: Distinguishes if this is a module or a theme.
- `core_version_requirement`: Indicates which Drupal versions are supported (e.g., `^9` or `^9 || ^10`). *Note: For Drupal 8.7.7 and earlier, the key was simply `core`.*

Common Optional Keys:

- `description`: helpful text for site administrators explaining what the module does.
- `package`: Groups modules together on the Extend page (e.g., "Custom", "Development").
- `dependencies`: A list of other modules required for this one to function.

```
name: 'My First Custom Module'
type: module
description: 'Code to spark my module development understanding.'
core_version_requirement: ^9 || ^10
package: 'Custom'
```

Listing 5: Example `mymodule.info.yml` file

4.3 The .module File

If your module needs to use legacy Drupal hook functions, you must also create a `[machine_name].module` file (e.g., `mymodule.module`). Even if it is empty initially, creating it prepares the module for procedural code. Once these files are in place, the module can be enabled from the Drupal UI under the Extend tab.

Unit 4 Summary

To define a new module, create a folder in `modules/custom/` and place an `info.yml` inside it. At a strict minimum, this YAML file must define the `name`, `type` (`module`), and `core_version_requirement`.

5 Unit 5: The Hook System and Implementing Hooks

5.1 What are Hooks?

Hooks are a fundamental feature of Drupal. Simply put, hooks are **functions that alter the behavior of Drupal core**. They are extremely powerful and allow you to provide, change, and work with various data at specific points in time during Drupal's processes (e.g., modifying entity data before it is saved, or changing a form structure before it is displayed).

Your custom modules can implement hooks defined by:

- Drupal Core
- Contributed Modules
- Other Custom Modules
- Your module can even define its own custom hooks!

5.2 The Three Categories of Hooks

Speaking broadly, there are over 270 hooks provided by Drupal core, which fall into three categories:

1. **Action Hooks (React to an action):** Invoked when specific events occur in the system, allowing other code to react.
 - *Example:* `hook_cron()` runs when the cron system operates.
2. **Alter Hooks (Alter existing data):** Invoked to allow modules to alter previously gathered data before it is fully processed or rendered. They are easy to identify because their function names literally end with `_alter`.
 - *Example:* `hook_form_alter()` allows you to modify the structure of forms.
3. **Information Hooks (Provide info):** Invoked when a component in Drupal is gathering a list of information about a particular topic.
 - *Example:* `hook_help()` provides details to Drupal to display on the system Help page.

5.3 five-step Process for Implementing a Hook

When you want to implement a hook in your custom module, follow these five steps carefully:

1. **Locate Documentation:** Find the hook documentation either by searching the `*.api.php` files in the codebase, or preferably, by viewing Drupal's API reference site (api.drupal.org/api/drupal/hooks).
2. **Copy Function Definition:** Copy the function declaration from the documentation and paste it into your `mymodule.module` file. Make sure to copy the correct parameters in the correct order.
3. **Rename the Function:** Substitute the word `hook` with your module's machine name (e.g., change `hook_cron` to `mymodule_cron`).
4. **Edit Documentation:** Add a docblock above the function. Per coding standards, this should be a single line: `Implements hook_[name_of_hook]`.
5. **Develop Functionality:** Write the PHP code to do what your module needs to do.

5.4 Implementation Example: `hook_cron`

Because a `.module` file is PHP, it must begin with the opening `<?php` tag, followed by an `@file` docblock describing the file.

```
<?php

/**
 * @file
 * This file demonstrates how to use hook functions.
 */

/**
 * Implements hook_cron().
 */
function mymodule_cron() {
  \Drupal::messenger()->addMessage('Hello from my module cron
    hook :)' );
}
```

Listing 6: Implementing `hook_cron` in `mymodule.module`

Note: Always flush the Drupal cache after adding or changing hook function definitions so Drupal registers them.

5.5 Implementation Example: `hook_form_alter`

This is a more complex hook that alters existing data. Notice that the `$form` array is passed by reference (using the `&` symbol), allowing us to directly modify it.

```
/**
 * Implements hook_form_alter().
 */
```

```
function mymodule_form_alter(&$form, FormStateInterface
    $form_state, $form_id) {
    // Target only the Article create/edit form
    if ($form_id == 'node_article_edit_form') {
        // Change the text of the submit button
        $form['actions']['submit']['#value'] = t('Save this awesome
            article');
    }
}
```

Listing 7: Implementing hook_form_alter to change a button text

Determining the correct Form ID: Finding the exact `$form_id` string can be tricky. There are two main methods:

1. **Examine HTML:** Inspect the form element in your browser. Look for the `id` attribute of the `<form>` tag (e.g., `node-article-edit-form`). Convert the dashes to underscores to get the required Drupal form ID (`node_article_edit_form`).
2. **Print Debug Message:** Temporarily add code at the top of your `hook_form_alter()` to print out all form IDs loaded on the page to the screen using the messenger service.

Unit 5 Summary

Hooks grant massive power to alter Drupal core behaviors. They fall into three main types: Action, Alter, and Information hooks. To implement them, find them in the Drupal API documentation, copy the signature to your `.module` file, replace "hook" with your module's machine name, and flush the cache!

6 Unit 6: Routing and Controllers

6.1 The Two-Step Design Approach

When building functionality in Drupal, a reliable design approach is:

1. **Build the tool:** Write the PHP code extending a base class or interface (e.g., creating a Controller).
2. **Wire it up:** Tell Drupal about it so it can be accessed (e.g., creating a Route).

6.2 Step 1: Building a Controller (PSR-4 Pattern)

A Controller is a piece of code that generates the output for a Response object. Standard practice is to place controller files in the `src/Controller/` directory of your module.

```

<?php

namespace Drupal\mymodule\Controller;

use Drupal\Core\Controller\ControllerBase;

/**
 * An example controller.
 */
class FirstController extends ControllerBase {

    /**
     * Returns a simple page.
     */
    public function simpleContent() {
        return [
            '#type' => 'markup',
            '#markup' => $this->t('Hello Drupal World!'),
        ];
    }
}

```

Listing 8: Creating src/Controller/FirstController.php

Note: The class name and file name must match exactly (`FirstController`), and they must be camel-cased. The namespace defines where Drupal's PSR-4 autoloader goes looking for the file.

6.3 Step 2: Wiring it with a Route (routing.yml)

To make the controller accessible via a URL, you must create a routing file named `[machine_name].routing.yml` in your module's base directory.

Required Route Elements:

- **path:** The URL path users visit. It must start with a forward slash (e.g., `/my-custom-path`).
- **defaults:** Maps the route to the specific controller method and can provide page defaults like `_title`.
- **requirements:** Specifies access controls, such as requiring a specific permission (e.g., `_permission: 'access content'`).

```

mymodule.simplecontent:
  path: '/my-custom-path'
  defaults:
    _controller: '\Drupal\mymodule\Controller\FirstController::
      simpleContent'
    _title: 'Simplicity is the ultimate sophistication'
  requirements:

```

```
_permission: 'access content'
```

Listing 9: Example mymodule.routing.yml file

Note: The route machine name (e.g., mymodule.simplecontent) should always begin with your module's machine name to ensure uniqueness.

6.4 Passing Variables Through Routes

You can pass dynamic arguments from the URL directly into your controller method by wrapping elements of the path in curly braces.

In routing.yml:

```
path: '/my-custom-path/{name_1}/{name_2}'
```

In FirstController.php:

```
public function variableContent($name_1, $name_2) {  
  return [  
    '#type' => 'markup',  
    '#markup' => $this->t('Hello @name_1 and @name_2', [  
      '@name_1' => $name_1,  
      '@name_2' => $name_2  
    ]),  
  ];  
}
```

Security Note: Always use the Translation system (`$this->t()`) with `@` placeholders when outputting variable user input to sanitize the strings and prevent SQL injection or cross-site scripting (XSS) attacks.

Unit 6 Summary

To create custom pages in Drupal, first build a **Controller** class inside the `src/Controller` directory extending `ControllerBase`. Then, wire it up using a **routing.yml** file that maps a specific path to that Controller's method, while enforcing access requirements.

7 Unit 7: Menu API and Menu Items

7.1 Defining Custom Menu Links

In Drupal, custom menu links are defined programmatically using YAML configuration files. To add a standard link to a menu (like the Main Navigation), you create a `[machine_name].links.menu.yml` file in the root of your custom module directory.

Key Components of a Menu Link YAML Definition:

- **Configuration Name:** Must be strictly unique, ideally formatted as `[module_name].[identifier]`.
- **title:** The text displayed for the link in the menu.

- **description:** Text shown as a tooltip when hovering, or as a description in the admin UI.
- **route_name:** The machine name of the route the link points to (e.g., `mymodule.simplecontent`). *Crucially, you use the route name here, not the URL path.*
- **weight:** Determines the display order. Drupal supports weights from -50 to 50. Lower numbers have higher priority and display first (e.g., -5 displays before 10).
- **menu_name:** Specifies which menu the link belongs to (e.g., `main` for Main Navigation).

```
mymodule.simplecontent_page:
  title: 'Simple Content'
  description: 'Link to the simple content method'
  route_name: mymodule.simplecontent
  weight: 10
  menu_name: main
```

Listing 10: Example `mymodule.links.menu.yml` file

Note: Menu links provided by modules via YAML configuration are "locked." The title and path cannot be edited by site content editors through the administrative UI.

7.2 Four Types of Menu Items

Drupal uses different YAML files to handle different types of links and tasks within the system:

1. **Normal Menu Items** (`links.menu.yml`): Standard links shown in menus (like the Main menu) and breadcrumbs. (This is what we created above).
2. **Local Tasks** (`links.task.yml`): Typically rendered as *tabs* on a page (e.g., View, Edit, Delete). They are local to the current route/page being viewed.

Local Task Menu Items

- Tabs local to the page
- **File:** `*.links.task.yml`

Welcome to Module Development

View Edit Delete Revisions

Submitted by courseadmin on Tue, 08/04/2020 - 00:01

All the world is a laboratory to the inquiring mind.

Add new comment

Subject

Comment *

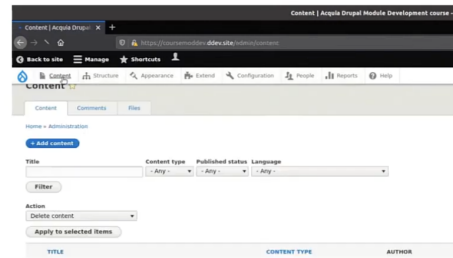
B I Format Source

<https://www.drupal.org/docs/drupal-apis/menu-api/providing-module-defined-local-tasks>

3. **Local Actions** (`links.action.yml`): Intended to trigger an action rather than just display information. Commonly rendered as a button link (e.g., the "Add Content" button on the administration page).

Local Action Menu Items

- Button link to invoke action
- **File:** *.links.action.yml

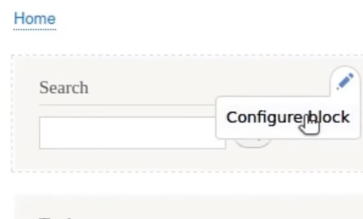


<https://www.drupal.org/docs/8/api/menu-api/providing-module-defined-local-actions>

4. **Contextual Links** (links.contextual.yml): Requires the core Contextual Links module. These are the quick-edit dropdown links that appear when clicking the pencil/gear icon next to an element (like a Block or Node) on the frontend.

Contextual Menu Items

- Links displayed when clicking the Gear
- **File:** *.links.contextual.yml



<https://www.drupal.org/docs/8/api/menu-api/providing-module-defined-contextual-links>

Unit 7 Summary

Menu items are defined cleanly via YAML files named according to their purpose (e.g., links.menu.yml for standard links). Instead of hardcoding URL paths, you assign the `route_name` to link the menu item to your custom Controller logic.

8 Unit 8: Building a Mini Module (RSVP List)

8.1 Sub-unit 1: Setting up the info.yml

To consolidate our learning, we are building a mini-module called **RSVP List**. Just like our previous custom module, we start by creating a directory (`/modules/custom/rsvplist`) and an `info.yml` file. The machine name of the folder is one word: `rsvplist`.

```
name: 'RSVP List'
type: module
description: 'Allows users to RSVP to events.'
package: 'Custom'
core_version_requirement: ^9 || ^10
dependencies:
  - drupal:block
configure: rsvplist.admin_settings
```

Listing 11: Example rsvplist.info.yml file

8.1.1 New info.yml Keys

In building this module, we introduce two important new keys to the `info.yml` file:

- **dependencies:** A list of other modules that must be enabled for this module to function.
 - They must be namespaced in the format `[project_name]:[module_name]`.
 - The `[project_name]` is the machine name of the project as it appears in the Drupal.org URL (e.g., `drupal.org/project/token`).
 - The `[module_name]` is the machine name of the module itself.
 - Example: Because RSVP List relies on Drupal Core's `block` module, which is part of the `drupal` project, it is listed as `drupal:block`. If it depended on the contributed Token module, it would be `token:token`. Note the preceding hyphen indicating a list item.
- **configure:** The route name to the module's main configuration form. A "Configure" link will appear next to the module on the Extend page (where you enable modules) once it is enabled. We specify `rsvplist.admin_settings` here, and we will build the route matching this name in a later unit.

8.1.2 YAML Syntax and Strings

A quick note on writing strings in YAML files:

- **Plain Scalars:** Technically, in YAML, strings are known as "plain scalars". You do not need to wrap strings in quotes (single `'` or double `"`) unless the string begins with or contains a YAML special character.
- **Special Characters:** There are around 20 special characters in YAML, including `@` (at sign), `|` (pipe sign), and `!` (exclamation point). For instance, starting a description with `@` without quotes would break Drupal's YAML parser.
- **Best Practice:** The caret (`^`) used in `core_version_requirement: ^9 || ^10` is *not* considered a special character by YAML, which is why it works without quotes. However, as a general rule of thumb, if you are unsure or if the string contains special characters, always wrap it in quotation marks.

Note: After creating this file and enabling the module in the "Extend" area, it will not do anything yet, but the module is officially installed.

Unit 8 Summary

When building a module that requires other modules to function, use the `dependencies` key structured as `project:module`. Use the `configure` key to link the administrative UI directly to your module's settings form route. Avoid starting YAML strings with special characters unless wrapped in quotes.

8.2 Sub-unit 2: The Form API

8.2.1 What is a Drupal Form?

In Drupal, a form encompasses three components executed in a sequential process:

1. **Display:** Drupal provides the actual HTML form rendered in the browser for collecting user input.
2. **Validation:** Drupal validates the user-entered data (e.g., if the user supplies an email, validation checks if it is formatted correctly). If it doesn't seem valid, Drupal presents an error message until the user provides valid data.
3. **Submission:** If the data is valid, Drupal handles the submission by determining the logic of what happens on the server to process the user input data.

Handling these tasks is simplified by the **Drupal Form API**, which operates as an abstraction layer around standard HTML forms. If you have manually created forms in HTML before, you know there is a laborious amount of coding involved due to redundancy. The Form API achieves consistency in form processing and presentation while reducing the HTML that must be explicitly generated. It controls the lifecycle of a form (definition, display, validation, submission) and provides helper utilities to sanitize inputs, handle POST data, and transliteration.

The Form API already has all the HTML form elements pre-defined (submit buttons, check boxes, select lists, text fields, date fields, etc.), making form building much less time-consuming.

8.2.2 Render Arrays

Essentially, forms are described with multi-dimensional arrays (an array of arrays). This associative array structure lets Drupal know exactly how a form should be rendered. In fact, Drupal Form Arrays are leveraging **Render Arrays**.

Render arrays are the building blocks of a Drupal page, conforming to the structural standards of Drupal's Render API (which integrates with the Theme API). We previously used simple render arrays in Unit 2 when our controller returned an array with `#type` and `#markup`. With forms, beyond `#markup`, you use Form API elements like check boxes and text fields.

Note: While Render APIs and Form APIs share elements and property structures, many properties on form elements only have meaning for the Form API. Practically, this

means you **MUST** declare your form arrays in the correct place in code (the `buildForm` method), or they will not have meaning to other systems.

8.2.3 Three Common Types of Forms

Specifically, there are three types of forms commonly used, and each has a base class you extend in your custom module:

1. **FormBase**: The most generic base class for generating standard forms. The other two form types actually extend this class. In the RSVP module, the form that collects visitor details extends `FormBase`.

Types of Forms

1. FormBase
2. ConfigFormBase
3. ConfirmFormBase

RSVP to this Event

Email address *

We'll send updates to the email address you provide.

RSVP

2. **ConfigFormBase**: Specialized for creating system configuration forms used by site administrators. In the RSVP module, the administrative settings form extends `ConfigFormBase`.

Types of Forms

1. FormBase
2. ConfigFormBase
3. ConfirmFormBase

FRONT PAGE

Default front page

https://www.drupal.org/ Home

Optionally, specify a relative URL to display as the front page. Leave blank to display the default front page.

ERROR PAGES

Default 403 (access denied) page

This page is displayed when the requested document is denied to the current user. Leave blank to display a generic "access denied" page.

Default 404 (not found) page

This page is displayed when no other content matches the requested document. Leave blank to display a generic "page not found" page.

Save configuration

3. **ConfirmFormBase**: Provides users with a form to confirm an action (like deleting a piece of content).

Types of Forms

1. FormBase
2. ConfigFormBase
3. ConfirmFormBase

Are you sure you want to delete this content item?

Home » Administration » Content

Welcome to Drupal Module Development

This action cannot be undone.

Delete

Cancel

8.2.4 Required Form Methods

With each of the three form types, there are three required methods you must implement:

1. `getFormId()`: Returns a string that serves as the unique ID of the form.
2. `buildForm()`: Returns the Form API array that defines each of the elements. This is the "correct place in code" mentioned earlier where your render arrays actually have meaning.
3. `submitForm()`: This is where you begin to do things with the valid data, such as saving it to the database or sending an email.

Note: The `validateForm()` method—which prevents submission if data is invalid—is technically not required by the interface, but it is highly recommended as a good default.

Unit 8 Sub-unit 2 Summary

The Form API abstracts HTML form creation using Render Arrays in a Display → Validate → Submit lifecycle. You must define these Render Arrays inside the `buildForm()` method of a class extending `FormBase`, `ConfigFormBase`, or `ConfirmFormBase`, alongside `getFormId()` and `submitForm()`.

8.3 Sub-unit 3: Building the Email Submission Form

Now we will apply the Form API concepts to build the actual `RsvpForm` class. Custom forms must be placed in the `src/Form` directory of your module.

For the RSVP module, create the file at: `/modules/custom/rsvpplst/src/Form/RsvpForm.php`.

Here is the complete code for the form, followed by an explanation of its components:

```
<?php

/**
 * @file
 * Contains \Drupal\rsvpplst\Form\RsvpForm.
 */

namespace Drupal\rsvpplst\Form;

use Drupal\Core\Form\FormBase;
use Drupal\Core\Form\FormStateInterface;

class RsvpForm extends FormBase {

  /**
   * {@inheritdoc}
   */
  public function getFormId() {
    return 'rsvpplst_email_form';
  }
}
```

```

}

/**
 * {@inheritdoc}
 */
public function buildForm(array $form, FormStateInterface
    $form_state) {
    // Attempt to get the fully loaded node object of the viewed
    // page.
    $node = \Drupal::routeMatch()->getParameter('node');

    // Some pages may not be nodes. Fail-safe to extract the node
    // ID.
    if (!is_null($node)) {
        $nid = $node->id();
    }
    else {
        $nid = 0;
    }

    $form['email'] = [
        '#type' => 'textfield',
        '#title' => $this->t('Email address'),
        '#size' => 25,
        '#description' => $this->t('We will send updates to the
            email address you provide.'),
        '#required' => TRUE,
    ];

    $form['submit'] = [
        '#type' => 'submit',
        '#value' => $this->t('RSVP'),
    ];

    $form['nid'] = [
        '#type' => 'hidden',
        '#value' => $nid,
    ];

    return $form;
}

/**
 * {@inheritdoc}
 */
public function submitForm(array &$form, FormStateInterface
    $form_state) {
    $submitted_email = $form_state->getValue('email');
    $this->messenger()->addMessage($this->t('The form is working!
        You entered @entry.', ['@entry' => $submitted_email]));
}

```

```
}
```

Listing 12: src/Form/RsvpForm.php

8.3.1 Code Explanation

- **Namespace & Uses:** Must end in Form (Drupal\rsvplist\Form) because of the PSR-4 directory structure. FormBase and FormStateInterface are required to build and submit the form.
- **Inheritdoc:** Notice {@inheritdoc} in the docblocks. Drupal coding standards mandate this exact tag (all lowercase, wrapped in braces) when implementing interface methods.
- **getFormId():** Returns the unique machine name of the form array (rsvplist_email_form).
- **buildForm():**
 - We use \Drupal::routeMatch()->getParameter('node') to dynamically grab the node being viewed (if any) and extract its ID for the hidden field.
 - We define three items in our render array: an email field (textfield), a submit button, and a nid field (hidden).
 - Labels/Values shown to users are wrapped in \$this->t() for translation.
- **submitForm():** The \$form_state object holds all the submitted values. We retrieve the email with getValue('email') and use the messenger service to print it to the screen (temporarily, until we add database logic).

Unit 8 Sub-unit 3 Summary

Custom forms are placed in src/Form. The buildForm() method defines the form's render array (text fields, hidden fields, submit buttons). You can use routeMatch() to dynamically pass contextual data (like the current Node ID) into hidden form fields, and use \$form_state->getValue() in the submitForm() method to retrieve user input.

8.4 Sub-unit 4: Building a Route with a Path to a Form

To test the RsvpForm we just built, we need a way to visit it in the browser. Even though this form will ultimately be displayed in a block, we can temporarily place the form on a dedicated page using Drupal's routing system to verify it works.

1. Create the Routing File

In the root directory of the RSVP List module (/modules/custom/rsvplist), create the routing file: rsvplist.routing.yml.

2. Define the Route

Since most routes share a common structure, you can copy an existing route (like the one we built in Unit 2 for `mymodule`) and modify its keys.

Here is the complete routing code:

```
rsvplist.form_demo:
  path: '/rsvplist-demo'
  defaults:
    _form: '\Drupal\rsvplist\Form\RsvpForm'
    _title: 'RSVP to this Event Demo'
  requirements:
    _permission: 'access content'
```

Listing 13: `rsvplist.routing.yml`

8.4.1 Code Explanation

Let's break down the specific changes made to support forms:

- **Route Name** (`rsvplist.form_demo`): The unique identifier for this route, following the standard convention of `[module_name].[identifier]`.
- `path`: The URL (`/rsvplist-demo`) users will visit in the browser to view the form.
- `_form` **Key**: This is the most crucial difference compared to a standard page route. Instead of using the `_controller` key to trigger a page callback, we use `_form`.
 - This tells Drupal that the destination is a Form API class, not just a standard controller returning a render array.
 - The value points directly to the fully-namespaced class we created: `\Drupal\rsvplist\Form`
 - **Important Note**: Unlike controllers where you must specify the method using double colons (e.g., `::build`), you *never* append a method when using `_form`. Drupal automatically knows to call the required methods (`getFormId`, `buildForm`, etc.) sequentially.
- `_title`: The title displayed on the page and in the browser tab (RSVP to this Event Demo).

8.4.2 Testing the Form

Once the routing file is saved:

1. Clear/flush the Drupal cache so the routing system registers the new `rsvplist.routing.yml` file.
2. Navigate to `/rsvplist-demo` in your browser.
3. You should see the fully rendered HTML form. Submit a test string (like "hello") to ensure the `submitForm()` method displays the confirmation message on the screen. (Note: It will accept "hello" because we haven't implemented email validation yet—that comes next!).

Unit 8 Sub-unit 4 Summary

To route to a form instead of a generic page, use the `_form` key under `defaults` in your `.routing.yml` file instead of `_controller`. The value must be the fully namespaced class trajectory (e.g., `\Drupal\module\Form\Class`) with no specific method explicitly called.

8.5 Sub-unit 5: Validating Form Submissions

Now that the form is routing and displaying correctly, we need to add a validation handler to ensure users only submit properly formatted emails.

We do this by injecting the `validateForm()` method directly into our `RsvpForm.php` class file, placing it above the `submitForm()` method:

```
/**
 * {@inheritdoc}
 */
public function validateForm(array &$form, FormStateInterface
    $form_state) {
    // 1. Get the submitted email value.
    $value = $form_state->getValue('email');

    // 2. Check if the value is a valid email using Drupal's
    email validator service.
    if (!\Drupal::service('email.validator')->isValid($value)) {
        // 3. If NOT valid, set an error on the 'email' field.
        $form_state->setErrorByName('email',
            $this->t('It appears that %mail is not a valid email.', [
                '%mail' => $value]));
    }
}
```

Listing 14: Adding Validation to `RsvpForm.php`

8.5.1 Code Explanation

- **Execution Order:** `validateForm()` runs *before* `submitForm()`.
- The `$form_state` variable contains all user-submitted data. We grab the email string using `$form_state->getValue('email')`.
- We invoke the core Drupal service `email.validator` and run its `isValid()` method on our mapped value.
- If it is **not** valid (hence the `!` negation operator), we call `$form_state->setErrorByName()` targeting the `email` field. This halts the submission and displays the custom error message.
- *Note:* `validateForm` shouldn't modify data or return anything; it only sets errors on `$form_state`. If any errors are set, Drupal halts the process and `submitForm` is never called.

Unit 8 Sub-unit 5 Summary

To validate form data, implement the `validateForm()` method within your form class. Extract values using `$form_state->getValue()`, run your checks, and if validation fails, use `$form_state->setErrorByName()` to halt submission and display an error.

8.6 Sub-unit 6: The Install File and Working with the Database Schema

Before we can use the Database API to insert records, we need tables to insert them into. The RSVP List module needs two tables: 1. `rsvplist`: Records details of the RSVPs (ID, User ID, Node ID, Email, Timestamp). 2. `rsvplist_enabled`: Tracks which specific nodes have the RSVP feature enabled by administrators.

8.6.1 The Schema API

Drupal's **Schema API** allows modules to declare their database tables safely using a structured multi-dimensional array, much like how the Form API defines forms. Hooking into this API is done via `hook_schema()`, which **must** be placed inside a `.install` file.

Tables declared by `hook_schema()` are automatically generated when the module is installed and subsequently dropped when the module is uninstalled.

8.6.2 Creating `rsvplist.install`

In the root directory of your module, create `rsvplist.install`. Since it is a PHP file, start with the `<?php` tag.

```
<?php
/**
 * @file
 * Install, update and uninstall functions for the rsvplist
 * module.
 */

/**
 * Implements hook_schema().
 */
function rsvplist_schema() {
  $schema['rsvplist'] = [
    'description' => 'Stores email, timestamp, nid and uid for an
      RSVP.',
    'fields' => [
      'id' => [
        'description' => 'The primary identifier for the record.'
        ,
        'type' => 'serial',
        'unsigned' => TRUE,
        'not null' => TRUE,
```



```

    ],
    'uid' => [
      'description' => 'The {users}.uid that added this RSVP.',
      'type' => 'int',
      'not null' => TRUE,
      'default' => 0,
    ],
    'nid' => [
      'description' => 'The {node}.nid for this RSVP.',
      'type' => 'int',
      'unsigned' => TRUE,
      'not null' => TRUE,
      'default' => 0,
    ],
    'mail' => [
      'description' => 'User\'s email address.',
      'type' => 'varchar',
      'length' => 64,
      'not null' => FALSE,
      'default' => '',
    ],
    'created' => [
      'description' => 'Timestamp for when RSVP was created.',
      'type' => 'int',
      'not null' => TRUE,
      'default' => 0,
    ],
  ],
  'primary key' => ['id'],
  'indexes' => [
    'node' => ['nid'],
    'node_user' => ['nid', 'uid'],
  ],
];

$schema['rsvplist_enabled'] = [
  'description' => 'Tracks whether RSVP is enabled or not for a
    node.',
  'fields' => [
    'nid' => [
      'description' => 'The {node}.nid that has RSVP enabled.',
      'type' => 'int',
      'unsigned' => TRUE,
      'not null' => TRUE,
      'default' => 0,
    ],
  ],
  'primary key' => ['nid'],
];

return $schema;

```

```
}
```

Listing 15: Creating tables with `hook_schema()`

8.6.3 Code Explanation

- **Array Structure:** The `$schema` array maps table names (`rsvplist`) to another array providing its definition.
- **Translation:** Notice that the `'description'` strings are *not* wrapped in `$this->t()` or `t()`. This text is written directly into the database schema table definitions and is not dynamic user-facing text.
- **Fields:** The `fields` key maps column names (e.g., `id`, `uid`) to their properties (`type`, `length`, `not null`).
- **Data Types:** Drupal uses generic data types (`serial`, `int`, `varchar`) which the Database Abstraction Layer converts into MySQL/PostgreSQL/MariaDB specific column types during creation. `serial` is an auto-incrementing integer.
- **Execution:** To see these tables appear in your database, you must go to the "Extend" page in the Drupal UI, **uninstall**, and then **re-install** the RSVP List module. `hook_schema()` only runs during installation.

Unit 8 Sub-unit 6 Summary

To create database tables, build a `.install` file and implement `hook_schema()`. Return a multi-dimensional associative array defining your table names, fields, data types, and primary keys. You must re-install the module for Drupal to execute this hook and generate the tables.

8.7 Sub-unit 7: An Introduction to the Database API

If you come from a background of writing native `mysql_query()` or `pg_query()` statements, Drupal's approach will feel different. The goal of the Drupal Database API (more accurately called the **Database Abstraction Layer**) is to allow developers to write *one* query object that works across multiple database types (MySQL, MariaDB, PostgreSQL).

Unless you are developing Drupal Core APIs, you should *never* use native PHP database functions to query the database. Behind the scenes, Drupal uses PDO (PHP Data Objects) for a consistent access method, translating your object-oriented queries into the appropriate vendor-specific SQL.

8.7.1 The Database Connection Object

Every database Drupal connects to has a single connection object. To get the active connection, use:

```
$connection = \Drupal::database();  
// Alternatively: \Drupal::service('database');
```

Listing 16: Getting the Database Connection

8.7.2 Static Queries vs. Dynamic Queries

Once you have the connection object, there are two styles of queries you can construct:

1. Static Queries

- Look very similar to raw SQL.
- Uses the `query()` method.
- **CRITICAL RULE:** Static queries should **only** be used for simple `SELECT` statements. *Never* use the `query()` method for `INSERT`, `UPDATE`, or `DELETE`.

```
$query = \Drupal::database()->query("SELECT id, example FROM {  
    cool_table}");
```

Listing 17: A Simple Static Select Query

2. Dynamic Queries

- Built step-by-step using object-oriented methods.
- Required for any complex `SELECT` and *all* `INSERT`, `UPDATE`, `DELETE`, and `MERGE` operations.
- Instead of using `query()`, you initiate dynamic queries using methods like `select()`, `insert()`, `update()`, `merge()` (`upsert`), or `delete()`.
- You chain methods like `fields()`, `condition()`, and `range()` to build the logic.
- **Execution:** The final step of any dynamic query is calling the `->execute()` method.

```
$query = \Drupal::database()->select('users_field_data', 'u');  
$query->fields('u', ['uid', 'name', 'created']);  
$query->condition('u.uid', 0, '<');  
$query->range(0, 50);  
$result = $query->execute();
```

Listing 18: A Dynamic Select Query

(Usually, dynamic queries are chained into a single line rather than declaring a variable for every step).

8.7.3 Handling Results and Errors

After executing a query, the result is an iterable object. You can loop through it using `foreach ($result as $record)`.

Alternatively, you can fetch the results into specific formats:

- `fetchAll()`: Returns an array of all result rows.
- `fetchCol()`: Returns a single column as an indexed array.
- `fetchAssoc()`: Returns the next row as an associative array.

Error Handling: Because databases can act unpredictably, it is highly recommended to wrap all database queries in a `try...catch` block. The Database API will throw an exception (`\Exception $e`) on error.

Unit 8 Sub-unit 7 Summary

Drupal's Database Abstraction Layer uses PDO to create vendor-agnostic queries. Access the connection with `\Drupal::database()`. Use **Static Queries** (`query()`) *only* for simple SELECTs. Use **Dynamic Queries** (`select()`, `insert()`, `update()`, `delete()`) for everything else. Always end dynamic queries with `execute()` and catch errors using `try...catch`.

8.8 Sub-unit 8: Dynamic Insert Queries - Inserting into the Database Programmatically

Now that we understand the Database Abstraction Layer, we can update the `submitForm()` method in `RsvpForm.php` (as shown in the code block in Sub-unit 3) to actually write user data to the database instead of just printing a test message.

8.8.1 The Three Phases of Form Submission

A good practice when handling form submissions bound for the database is to wrap the logic inside a `try...catch (\Exception $e)` block. Databases can throw unexpected exceptions, and failing gracefully provides a better user experience. Within this block, the logic follows three phases:

Phase 1: Initiate Variables

We need four specific values to insert into the `rsvplist` table:

1. **UID (User ID):** Because the form doesn't explicitly ask for a User ID, we fetch the ID of the currently logged-in user programmatically: `\Drupal::currentUser()->id()`.
2. **NID and Email:** We extract these directly from the user's submission using the form state object: `$form_state->getValue('nid')` and `$form_state->getValue('email')`.
Note: As defined in `buildForm`, if the form isn't placed on a node yet (like our `/rsvplist-demo` route), the NID defaults to 0.

3. **Created Timestamp:** Instead of native PHP `time()`, we leverage Drupal's time service: `\Drupal::time()->getRequestTime()`.

Phase 2: Construct and Execute the Query

We build a dynamic query targeting the `rsvplist` table:

1. Initiate: `$query = \Drupal::database()->insert('rsvplist');`
2. Map fields: Pass an associative array to the `fields()` method where the array keys match the exact database column names, and the values are the variables from Phase 1.
3. Execute: `$query->execute();`

Phase 3: Provide User Feedback

Never leave the user guessing. Use the `messenger()` service to print a success message (or an error message in the `catch` block) using `$this->t()` for translation safety.

Unit 8 Sub-unit 8 Summary

Form submissions saving to a database should extract active variables (via `currentUser()`, `time()`, and `$form_state`), map them using a dynamic query (e.g., `insert()->fields()->execute()`), print feedback via the Messenger service, and wrap everything in a `try...catch` block for safety.

8.8.2 Updating the submitForm Code

With this understanding, we update the `submitForm()` method inside `RsvpForm.php` to dynamically insert the data into the database:

```
/**
 * {@inheritdoc}
 */
public function submitForm(array &$form, FormStateInterface
    $form_state) {
    try {
        // Phase 1: Initiate variables to save.
        $uid = \Drupal::currentUser()->id();
        $nid = $form_state->getValue('nid');
        $email = $form_state->getValue('email');
        $current_time = \Drupal::time()->getRequestTime();

        // Phase 2: Construct the Dynamic Insert Query and execute
        it.
        $query = \Drupal::database()->insert('rsvplist');
        $query->fields([
            'uid' => $uid,
            'nid' => $nid,
            'mail' => $email,
            'created' => $current_time,
        ]);
    }
```

```

$query->execute();

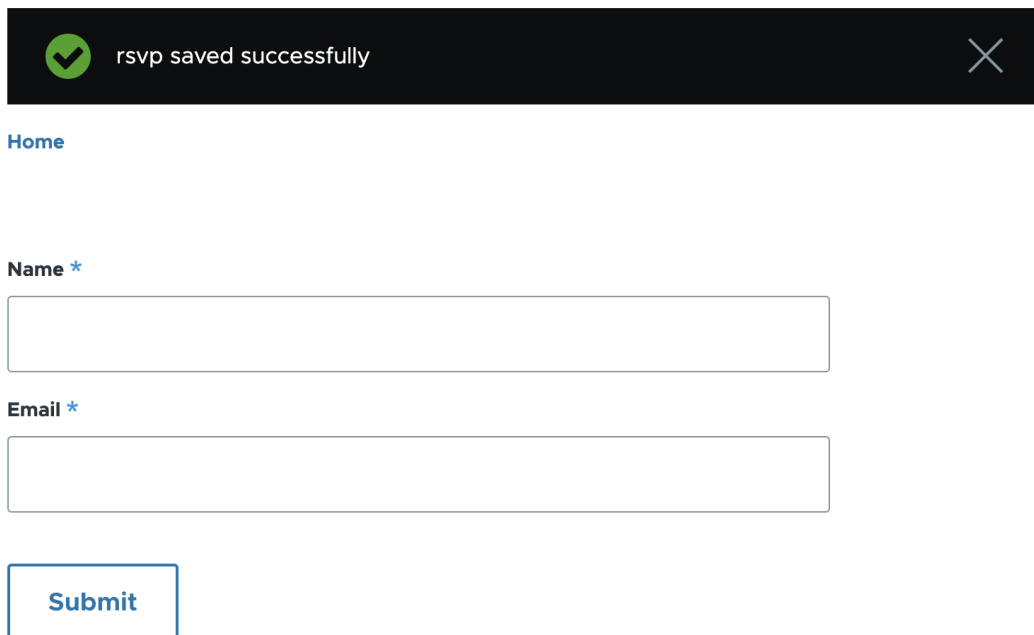
// Phase 3: Display a success message.
\Drupal::messenger()->addMessage(
    $this->t('Thank you for your RSVP, you are on the list
           for the event.')
);
}
catch (\Exception $e) {
    \Drupal::messenger()->addError(
        $this->t('Unable to save RSVP settings at this time due
                to database error. Please try again.')
    );
}
}

```

Listing 19: Updating submitForm() in RsvpForm.php

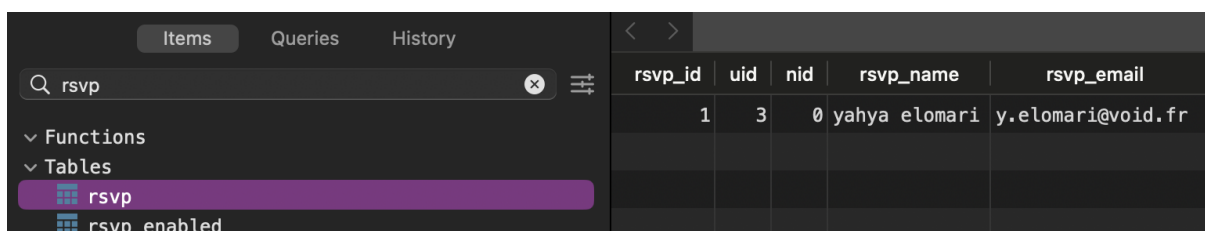
8.8.3 Validating the Insertion (Database Check)

After implementing the validation and dynamic insertion logic into submitForm(), submitting a valid email through the /rsvplist-demo form triggers the success message:



A screenshot of a web form interface. At the top, a dark green banner displays a checkmark icon and the text "rsvp saved successfully", with a close button (X) on the right. Below the banner, a link labeled "Home" is visible. The form contains two required input fields: "Name *" and "Email *", both of which are currently empty. Below these fields is a blue "Submit" button.

Checking the database directly (using tools like phpMyAdmin) confirms that Drupal successfully matched the fields and injected the record safely via PDO:



A screenshot of the phpMyAdmin interface. The left sidebar shows the database structure with "rsvp" selected. The main panel displays a table with the following data:

rsvp_id	uid	nid	rsvp_name	rsvp_email
1	3	0	yahya elomari	y.elomari@void.fr

8.9 Sub-unit 9: Defining Custom Permissions - Flexible Access Control

By default, we set our form route (`rsvplist-demo`) to require the generic access content permission. This means anyone (even anonymous visitors) can access it. To restrict access and provide granular control, we can define custom permissions specific to our module.

8.9.1 Creating permissions.yml

In the root directory of your module (`/modules/custom/rsvplist`), create the file `rsvplist.permissions.yml`.

Permissions are defined using simple YAML key-value pairs where the parent key is the **machine name** of the permission.

```
view rsvp list:
  title: 'View RSVP List'
  description: 'Allowed to view the RSVP list.'
access rsvp list report:
  title: 'Access RSVP List Report'
  description: 'Allowed to access the RSVP list report.'
administer rsvp list:
  title: 'Administer RSVP List'
  description: 'Allowed to administer the RSVP list settings.'
  restrict access: true
```

Listing 20: `rsvplist.permissions.yml`

Key Attributes:

- **Machine Name** (`view rsvp list`): Used in code/routing to strictly check access.
- **title**: The human-readable name displayed on the permissions administration page (`/admin/people/permissions`).
- **description** (Optional): Extra helper text for the site admin.
- **restrict access** (Optional): Set to `true` to display a warning to site admins indicating that this permission grants significant power (intended only for trusted roles).

8.9.2 Applying the Permission

Defining a permission does nothing until it is enforced. Open `rsvplist.routing.yml` and update the form route's requirements to use the new permission machine name:

```
requirements:
  _permission: 'view rsvp list'
```

Listing 21: Updating `rsvplist.routing.yml`

After flushing the Drupal cache, anonymous visitors attempting to visit `/rsvplist-demo` will seamlessly be met with an Access Denied message, proving our custom permission is working:

You are not authorized to access this page.

Unit 8 Sub-unit 9 Summary

Define custom permissions in a `.permissions.yml` file using machine names as the parent key. Secure routes by updating the `_permission` requirement in `.routing.yml` to match the new machine name. Assign the new permissions to user roles via the UI.

8.10 Sub-unit 10: Block Plugins and Creating the RSVP List Block

Currently, the RSVP form lives entirely on its own dummy page. In standard configurations, forms like this are displayed as **Blocks** so they can be placed in sidebars or footers across various pages.

Recall from Unit 3 that a Block is an instance of the Block Plugin type and is discovered by Drupal using PHP Annotations.

8.10.1 Creating the Plugin Directory and File

Following the strict PSR-4 standard for plugins:

1. Inside `src/`, create a `Plugin` directory.
2. Inside `src/Plugin`, create a `Block` directory.
3. Inside `src/Plugin/Block`, create the file `RsvpBlock.php`.

8.10.2 Coding the Block Plugin Class

The class must extend `BlockBase` and heavily relies on the `@Block` annotation snippet above the class declaration.

```
<?php

namespace Drupal\rsvplist\Plugin\Block;
```



```

use Drupal\Core\Block\BlockBase;

/**
 * Provides an 'RSVP' List Block.
 *
 * @Block(
 *   id = "rsvp_block",
 *   admin_label = @Translation("RSVP Block"),
 * )
 */
class RsvpBlock extends BlockBase {

  /**
   * {@inheritdoc}
   */
  public function build() {
    return [
      '#markup' => $this->t('My RSVP List Block'),
    ];
  }
}

```

Listing 22: src/Plugin/Block/RsvpBlock.php

8.10.3 Code Explanation

- **Namespace:** Explicitly marks the path where the autoloader looks: `Drupal\rsvplist\Plugin\B`.
- **Annotation (@Block):** This is how Drupal's Plugin Manager discovers the block without needing extra YAML files. It registers the system id (`rsvp_block`) and the translated `admin_label` used in the UI.
- **build() method:** Similar to a Controller, a block must have a `build()` method that returns a Render Array. For this initial test, we are just returning a static string of markup text. In a future unit, we will inject the actual form here.

Once the file is saved and the cache is flushed: 1. Navigate to **Structure** → **Block layout**. 2. Find the **Sidebar first** region and click **Place block**. 3. Search for "RSVP Block" (the `admin_label` from our annotation) and place it!

Unit 8 Sub-unit 10 Summary

Blocks are Plugins. Create them by placing a class within the `src/Plugin/Block/` namespace. The class must extend `BlockBase` and include the `@Block` docblock annotation (containing `id` and `admin_label`) for discovery. Finally, implement the `build()` method to return a Render Array of its contents.

8.11 Sub-unit 11: Displaying the RSVPForm in a Block and Applying Access Control

Now that the Block plugin is registered, it's time to replace the static dummy text with our actual `RsvpForm`. Additionally, we only want this block to appear if the user has the correct permissions and *only* if they are viewing a Node (since RSVPs apply to Nodes).

To achieve this, we will rely on three key Drupal functions: 1. `\Drupal::formBuilder()->getForm()`: Returns a renderable form array when passed the fully-namespaced name of a form class. 2. `AccessResult::allowedIfHasPermission()`: Returns an access result object determining if a user has a specific permission. 3. `blockAccess()`: A method inherited from `BlockBase` used to define custom access logic for the block displaying.

8.11.1 Updating the RSVP Block Plugin

We will edit `RsvpBlock.php` to implement these functions. We need to update the `build()` method and add the `blockAccess()` method.

```
<?php

namespace Drupal\rsvplist\Plugin\Block;

use Drupal\Core\Block\BlockBase;
use Drupal\Core\Session\AccountInterface;
use Drupal\Core\Access\AccessResult;

/**
 * Provides an 'RSVP' List Block.
 *
 * @Block(
 *   id = "rsvp_block",
 *   admin_label = @Translation("RSVP Block"),
 * )
 */
class RsvpBlock extends BlockBase {

  /**
   * {@inheritdoc}
   */
  public function build() {
    // Return the form render array instead of static markup.
    return \Drupal::formBuilder()->getForm('\Drupal\rsvplist\Form\RsvpForm');
  }

  /**
   * {@inheritdoc}
   */
  public function blockAccess(AccountInterface $account) {
    // If viewing a node, perform access check based on our
    // custom permission.
  }
}
```

```

    $node = \Drupal::routeMatch()->getParameter('node');

    if ($node) {
        return AccessResult::allowedIfHasPermission($account, 'view
            rsvp list');
    }

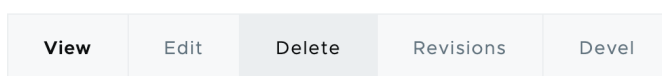
    // By default, if not on a node, deny access to the block.
    return AccessResult::forbidden();
}
}

```

Listing 23: Updating src/Plugin/Block/RsvpBlock.php

8.11.2 Code Explanation

- **Form Builder:** Instead of manually instantiating the form class, we ask the Form Builder service to do it for us using `getForm('\namespace\ClassName')`. We *do not* include `src` in the namespace due to PSR-4 autoloading rules.
- `blockAccess()`: This method automatically receives the current user's `$account` as an argument.
- **Route Matching:** We use `\Drupal::routeMatch()->getParameter('node')` to check if the current page is a Node. If it returns an object, we are on a node.
- **Access Results:** `allowedIfHasPermission()` cleanly checks if the provided `$account` has the 'view rsvp list' permission we created in Sub-unit 9. If the page is not a node at all (like the front page), the method falls back to returning `AccessResult::forbidden()`, ensuring the block never shows up where it shouldn't.



Welcome home

RSVP Form

Name *

Email *

Submit

Unit 8 Sub-unit 11 Summary

Use `\Drupal::formBuilder()->getForm()` to dynamically load and render forms inside blocks. Control block visibility by overriding the `blockAccess()` method, returning `AccessResult` objects based on route parameters (like checking for a node) and user permissions (`allowedIfHasPermission()`).

8.12 Sub-unit 12: Introduction to the Configuration API

As we prepare to build the Settings Form for site administrators (allowing them to choose which node types can have RSVPs), we need a reliable way to save those settings. This introduces the **Configuration API**.

The Configuration API provides a unified approach for modules to store configuration data.

8.12.1 Configuration vs. State

It is crucial to understand when to use Configuration and when to use the State API:

Configuration:

- Data that stays consistent and needs to be synchronized between site environments (Development → Staging → Production).
- Created during site building (e.g., automated cron settings, site name, block placements, views).
- Stored in YAML files during export, making it deployable.

State Variables:

- Local variables that should *not* travel between environments.
- Hidden system values, last execution times, or cached data.
- If you reset a system, you lose all State data, but Configuration remains.

8.12.2 Active Configuration and Storage

The configuration currently mapped by a Drupal site is called the **Active Configuration**. By default, the active configuration is stored in the database for speed, but the storage backend is configurable (it can be stored in files or other systems).

As a developer, the Configuration API abstracts this storage away. You interact with configuration objects the same way regardless of where they are physically stored.

8.12.3 Configuration Objects

Configuration is divided into individual **objects**, each with a unique name/key (e.g., `mymodule.settings`).

- **Data Types:** Values inside a config object can be integers, strings, booleans, or nested hierarchies known as mappings.
- **File Format:** When configuration is exported for deployment, it is saved as YAML. Each configuration file contains *one* configuration object.
- **Naming Convention:** The YAML file name perfectly matches the configuration object name with a `.yaml` extension (e.g., `rsvplist.settings.yaml`).

Here are examples of exported configuration viewed from the Drupal UI. A simple configuration object (like basic site settings):

Why Configuration API

- A central place to store configuration data.
- Data stays consistent between environments.
- **Simple Configuration**
 - Site name, RSVPList settings, etc.
- **Complex Configuration**
 - Content types, Views, Blocks, etc.

Configuration type

Simple configuration

Configuration name

automated_cron.settings

automated_cron.settings

comment.settings

contact.settings

core.extension

core.menu.static_menu_link_overrides

dblog.settings

devel.settings

devel.toolbar.settings

field.settings

field_ui.settings

file.settings

filter.settings

image.settings

menu_ui.settings

node.settings

rsvplist.settings

search.settings

seven.settings

system.authorize

system.cron

A more complex configuration object (like block placement with dependencies):

Why Configuration API

- A central place to store configuration data.
- Data stays consistent between environments.
- **Simple Configuration**
 - Site name, RSVPList settings, etc.
- **Complex Configuration**
 - Content types, Views, Blocks, etc.

Configuration type

Content type

Simple configuration

Action

Base field override

Block

Comment type

Contact form

Content type

Custom block type

Date format

Entity form display

Entity view display

Field

Field storage

Form mode

Image style

Menu

RDF mapping

Role

Search page

Shortcut set

Taxonomy vocabulary

Text Editor

Text format

Tour

View

View mode

Unit 8 Sub-unit 12 Summary

Use the **Configuration API** for settings that move between server environments (deployable via YAML). Use the **State API** for local, environment-specific data. The active configuration is usually stored in the database but abstracted via the API into distinct objects (e.g., `module.settings`).

8.13 Sub-unit 13: Creating RSVPSettingsForm and Config Settings

To make the RSVP List module more useful, we want site administrators to be able to choose which content types (e.g., Article, Basic Page) should have the RSVP feature enabled. We will build a Settings Form leveraging Drupal's Configuration API.

8.13.1 The ConfigFormBase Class

Unlike the generic `FormBase` we used earlier, configuration forms extend `ConfigFormBase`. This base class provides:

1. Built-in methods to `get()` and `set()` configuration via the API.
2. A default "Save configuration" submit button (no need to define one in the form array).
3. An additional required method: `getEditableConfigNames()`, which returns an array of the configuration objects this form is allowed to edit.

8.13.2 1. Creating the RsvpSettingsForm Classes

In `src/Form/RsvpSettingsForm.php`, we define our configuration form:

```
<?php

namespace Drupal\rsvplist\Form;

use Drupal\Core\Form\ConfigFormBase;
use Drupal\Core\Form\FormStateInterface;

class RsvpSettingsForm extends ConfigFormBase {

    public function getFormId() {
        return 'rsvplist_admin_settings';
    }

    protected function getEditableConfigNames() {
        return [
            'rsvplist.settings',
        ];
    }
}
```

```

public function buildForm(array $form, FormStateInterface
    $form_state) {
    // Get all content types.
    $types = node_type_get_names();

    // Retrieve active configuration using the API.
    $config = $this->config('rsvplist.settings');

    $form['rsvplist_types'] = [
        '#type' => 'checkboxes',
        '#title' => $this->t('The content types to enable RSVP
            collection for'),
        '#default_value' => $config->get('allowed_types'),
        '#options' => $types,
        '#description' => $this->t('On the specified node types, an
            RSVP option will be available and can be enabled while
            that node is being edited.'),
    ];

    return parent::buildForm($form, $form_state);
}

public function submitForm(array &$form, FormStateInterface
    $form_state) {
    $selected_allowed_types = array_filter($form_state->getValue(
        'rsvplist_types'));
    sort($selected_allowed_types);

    // Set configuration variables and save.
    $this->config('rsvplist.settings')
        ->set('allowed_types', $selected_allowed_types)
        ->save();

    parent::submitForm($form, $form_state);
}
}

```

Listing 24: src/Form/RsvpSettingsForm.php

Key takeaways from the code:

- `$this->config('rsvplist.settings')` loads the configuration object defined in `getEditableConfigNames()`.
- In `buildForm`, we `->get('allowed_types')` to set the default checked boxes.
- In `submitForm`, we chain `->set('allowed_types', $types)->save()` to write it back.
- `parent::buildForm` and `parent::submitForm` handle adding the save button and printing the success message automatically.

8.13.3 2. Creating Default Configuration

We must establish the default values for `rsvplist.settings` upon module installation.
Rule: Default configuration files must be placed exactly at `config/install/` in your module directory.

Create `config/install/rsvplist.settings.yml`:

```
allowed_types:
  - article
```

Listing 25: `config/install/rsvplist.settings.yml`

This simply means that upon installation, the 'Article' content type will be checked by default. *Note: This file is only read when the module is installed. To apply it, you must uninstall and reinstall the module.*

8.13.4 3. Defining the Configuration Schema

Drupal needs to understand the structure "schema" of your configuration so it can translate and type-cast it correctly during imports/exports.

Create `config/schema/rsvplist.schema.yml`:

```
rsvplist.settings:
  type: config_object
  label: 'RSVP List Settings'
  mapping:
    allowed_types:
      type: sequence
      label: 'Allowed node types'
      sequence:
        type: string
        label: 'Node type'
```

Listing 26: `config/schema/rsvplist.schema.yml`

This tells Drupal that `rsvplist.settings` is an object containing a mapping called `allowed_types`, which is a sequential array of strings.

Unit 8 Sub-unit 13 Summary

To build settings pages, define a form extending `ConfigFormBase`. Implement `getEditableConfigNames()` to declare the config objects you will edit. Use `$this->config('name')->get()` and `->set()->save()` to interact with data. Provide default config in `config/install/name.yml` and define its data structure in `config/schema/name.schema.yml`.

8.14 Sub-unit 14: Add Routing and Menu Link to RSVPSettings

Now that the `RsvpSettingsForm` is built, it needs a URL (route) so administrators can visit it, and a link in the administrative menu to make it discoverable.

8.14.1 1. Creating the Route

We will add a new route to `rsvplist.routing.yml`. *Note: If you still have the temporary demo route for the RSVP form, you can remove or replace it now.*

```
rsvplist.admin_settings:
  path: '/admin/config/content/rsvplist'
  defaults:
    _form: '\Drupal\rsvplist\Form\RsvpSettingsForm'
    _title: 'RSVP List Settings'
  requirements:
    _permission: 'administer rsvp list'
```

Listing 27: Updating `rsvplist.routing.yml`

- **Path:** We use `/admin/config/content/rsvplist` because this is an administrative setting related to content modeling.
- **Form:** Points directly to the new `RsvpSettingsForm` class.
- **Permission:** Requires the `administer rsvp list` permission to ensure only highly privileged users can change these settings.

8.14.2 2. Creating the Menu Link

To make the form accessible via the **Configuration** → **Content Authoring** menu in the Drupal UI, we must create a new file specifically for menu links.

Create a file named `rsvplist.links.menu.yml` in the root of your module directory:

```
rsvplist.admin_settings_link:
  title: 'RSVP List Settings'
  description: 'Configure which content types support RSVPs.'
  route_name: rsvplist.admin_settings
  parent: system.admin_config_content
```

Listing 28: `rsvplist.links.menu.yml`

- **route_name:** The exact machine name of the route we just created (`rsvplist.admin_settings`).
- **parent:** This dictates where the link appears in the menu hierarchy. `system.admin_config_content` is the machine name of the route for the main **Configuration** → **Content Authoring** page provided by Drupal core.

Tip for finding Parent Routes: If you know the URL you want to nest your link under (e.g., `/admin/config/content`), you can perform a global text search across Drupal Core for that string specifically inside `*.routing.yml` files (in this case, found in `core/modules/system/system.routing.yml`) to identify the correct route name.

After saving these files and flushing the cache, you can navigate to **Configuration** → **Content Authoring** → **RSVP List Settings**, enabling site editors to toggle which content types collect RSVPs!

localhost/en/admin/config/content

rk | Hamza sauce | week5-drupalMod... | gold mine | leetcode devops | week3-php

Shortcuts | admin | Go to | Devel

Home > Administration > Configuration

Content authoring ☆

> **Linkit**

Manage Linkit profiles.

> **Scheduler**

Configure settings for scheduled publishing and unpublishing.

> **Text formats and editors**

Select and configure text editors, and how content is filtered when displayed.

> **RSVP settings**

Settings for the RSVP module.

RSVP settings



Status message

The configuration options have been saved.

Node types

- ☒ Article
- ☒ News
- ☐ Basic page
- ☐ slider

Select the node types for which RSVP is enabled.

[Save configuration](#)

Unit 8 Sub-unit 14 Summary

Define the URL to your settings form in `module.routing.yml` (using an `_form` default). Then, place a link to that route into the administrative UI by defining it in `module.links.menu.yml`, assigning the appropriate parent route (like `system.admin_config_content`) so it sits logically in the config dashboard.

8.15 Sub-unit 15: Creating an RSVP List Reports Page

Site administrators need a way to view all the RSVP sign-ups. We will create a custom page to display this data in a tabular format. Since this page will primarily display information (not collect it via a form), we will use a **Controller**.

8.15.1 Creating the ReportController

Controllers belong in the `src/Controller` directory. Create a new file named `ReportController.php`:

```
<?php

namespace Drupal\rsvplist\Controller;

use Drupal\Core\Controller\ControllerBase;
use Drupal\Core\Database\Database;

class ReportController extends ControllerBase {

  /**
   * Gets all RSVPs for all nodes.
   *
   * @return array|null
   */
  protected function load() {
    try {
      // Begin dynamic select query on rsvplist table.
      $database = \Drupal::database();
      $select_query = $database->select('rsvplist', 'r');

      // Join the user config table to get the username.
      $select_query->join('users_field_data', 'u', 'r.uid = u.uid');
      // Join the node field data table to get the node title.
      $select_query->join('node_field_data', 'n', 'r.nid = n.nid');

      // Select specific fields for the report.
      $select_query->addField('u', 'name', 'username');
      $select_query->addField('n', 'title');
      $select_query->addField('r', 'mail');

      // Execute query and fetch all results into an array of
      // associative arrays.
      $entries = $select_query->execute()->fetchAll(\PDO::
        FETCH_ASSOC);
      return $entries;
    }
    catch (\Exception $e) {
      \Drupal::messenger()->addError(t('Unable to access the
        database at this time. Please try again later.'));
      return NULL;
    }
  }

  /**
   * Creates the report page.
   */
}
```

```

* @return array
*   Render array for report output.
*/
public function report() {
    $content = [];
    $content['message'] = [
        '#markup' => $this->t('Below is a list of all Event RSVPs
        including username, email address and the name of the
        event they will be attending. '),
    ];

    $headers = [
        t('Username'),
        t('Event'),
        t('Email'),
    ];

    // Load data utilizing our protected load() method.
    $table_rows = $this->load();

    // Create the render array for a table.
    $content['table'] = [
        '#type' => 'table',
        '#header' => $headers,
        '#rows' => $table_rows,
        '#empty' => t('No entries available. '),
    ];

    // Do not cache this page by setting the max-age to 0.
    $content['#cache']['max-age'] = 0;

    return $content;
}
}

```

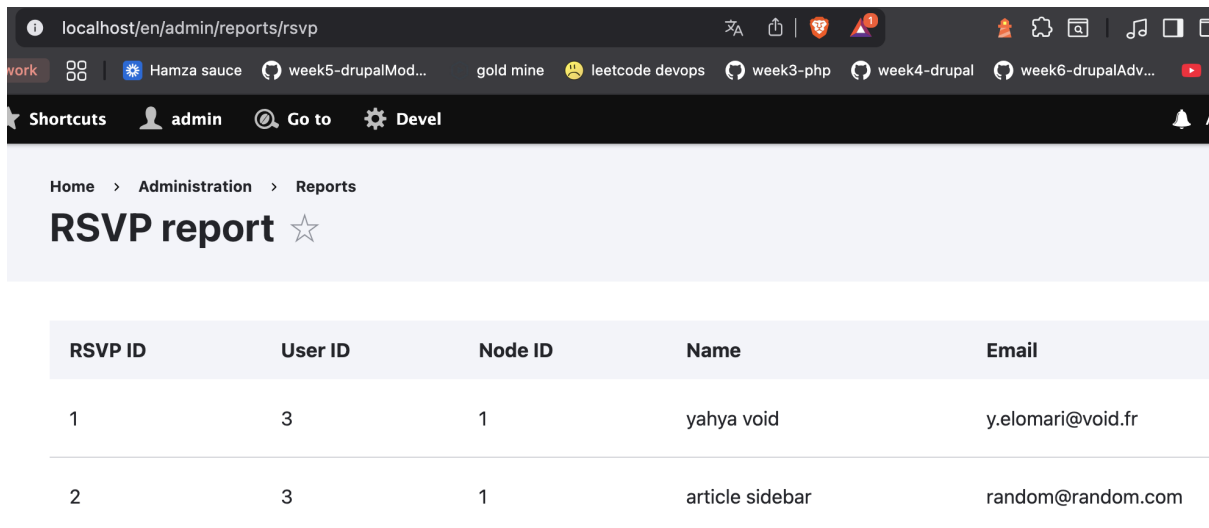
Listing 29: src/Controller/ReportController.php

8.15.2 Code Explanation

- **Dynamic Select Queries with Joins:** Inside the `load()` method, we build a dynamic query via `$database->select('rsvplist', 'r')`. We then `join()` two core Drupal tables (`users_field_data` and `node_field_data`) so we can grab the actual username strings and node titles instead of just their raw ID numbers.
- **Fetching Results:** `fetchAll(\PDO::FETCH_ASSOC)` is highly efficient here. Because the fields we selected map exactly to the headers we plan to display, we can fetch the entire database response directly into a ready-to-use array of arrays.
- **Table Render Array:** Inside the `report()` method, we return a Render Array of type `table`. We pass it our `$headers` mapping array and our raw `$table_rows`

data array. Drupal's theme engine handles rendering the actual HTML `<table>` tags.

- **Cache Invalidation:** `$content['#cache']['max-age'] = 0;` prevents Drupal from caching the output of this controller. Because RSVP data is dynamic and frequently updated, we force the page to render fresh every time it is visited.



The screenshot shows a web browser at `localhost/en/admin/reports/rsvp`. The breadcrumb trail is `Home > Administration > Reports`. The page title is `RSVP report` with a star icon. Below the title is a table with the following data:

RSVP ID	User ID	Node ID	Name	Email
1	3	1	yahya void	y.elomari@void.fr
2	3	1	article sidebar	random@random.com

Unit 8 Sub-unit 15 Summary

Use **Controllers** for standard pages that output data. Build dynamic select queries with `join()` to combine relational data from different database tables. Structure output utilizing the `table` Render Array type alongside defined `#header` and `#rows` properties. Disable caching for highly dynamic content using `#cache max-age = 0`.